Architecture Review — `refinery_process_model`

Plain-English-first edition. Generated: 2026-07-01.

Candidate A — Make `run_scenario` the "one big button"

Strong

Separate "do the calculations" from "find and load files".

Plain English summary

Right now, running the model often depends on knowing where the TOML/Excel files live and passing a repo folder around. This change would make one main function you call with inputs, and it would return results—without you needing to care about file paths.

Why you care

- Fewer "mystery requirements" (like "must run from this folder").
- Easier to run lots of scenarios (change inputs, rerun).
- Tests can run faster because they don't need to read files.

What I would actually do

1. Create a small "inputs loader" object that knows how to read TOML/Excel from disk.
2. Make `run_scenario` accept already-loaded inputs (Python dicts / dataclasses), not file paths.
3. Keep a separate function like `run_scenario_from_repo(base_dir, config)` that does the file loading and then calls `run_scenario`.
4. Update regression tests to call `run_scenario` with in-memory inputs when possible.

Files

- `refinery_process_model/core.py`
- `refinery_process_model/io_inputs.py`
- `refinery_process_model/paths.py`

Problem

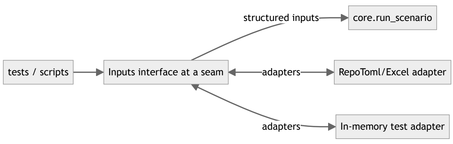
The **module** `core.run_scenario` aims to be a deep interface, but still mixes algorithmic work with repo-specific file loading and path resolution. This reduces **locality**: tests and callers need to know about TOML layout and `base_dir`.

Solution

Create a **seam**: an **interface** for "provide inputs", with an **adapter** for repo TOML/Excel loading and an **adapter** for in-memory test data. Keep `run_scenario` mostly pure.

Before

After



Benefits

- **Locality:** file-layout knowledge concentrates in one adapter.
- **Leverage:** the interface buys many scenario runs without repeating setup.
- **Tests:** the interface is the test surface; fewer slow, fragile IO tests.

Candidate B — Stop passing "giant dictionaries" between steps

Worth exploring

Concentrate key names and ordering into one Pipeline module.

Plain English summary

Several steps hand each other big dictionaries (tables) and you have to remember exact key names like `outputs["BPCD"]["Kerosine"]`. This change would put that "key knowledge" in one place.

Why you care

- Fewer bugs from typos or inconsistent naming (Kerosine/Kerosene).
- Easier to add a new output without editing many files.
- Easier to understand the workflow end-to-end.

What I would actually do

1. Create one "Pipeline" function/class that runs steps in order.
2. Inside Pipeline, convert messy dict outputs into a small, named Python object.
3. Update downstream steps to take the named object instead of raw dicts.
4. Add 2–3 tests that call Pipeline and check a few key outputs.

Files

- `refinery_process_model/distillation_sl`
- `refinery_process_model/hydrotreatment_`
- `refinery_process_model/cost_inputs.py`

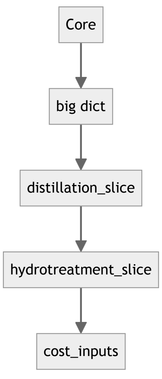
Problem

Slice **modules** accept/return wide dict tables. The **interface** is nearly as complex as the **implementation**, so the modules are shallow and reduce **leverage**.

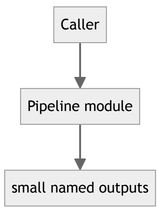
Solution

Add a Pipeline **module** that owns ordering + normalization. Keep slices behind the Pipeline's **seam** so callers don't need table keys.

Before



After



Benefits

- **Locality:** key-munging lives in one place.
- **Leverage:** a smaller interface gives more behavior per concept learned.
- **Tests:** fewer brittle tests asserting dict keys everywhere.

Candidate C — Make "testing the model" one simple command

Speculative

Replace many run-by-hand scripts with a consistent test runner.

Plain English summary

Today, "tests" are mostly separate Python scripts you run one at a time. This would make testing a single command that gives a clear report of what passed/failed.

Why you care

- Easier to check "did I break anything?" after edits.
- Clearer failures (which part broke, with a stack trace grouped nicely).
- Ability to run only fast tests vs slow tests.

What I would actually do

1. Pick one test tool (usually `pytest`).
2. Move shared setup into a single place (fixtures).
3. Convert 2–3 of the current script-tests into real tests first.
4. Add a short README: "run tests with this command".

Files

- `refinery_process_model/tests_*.py`
- `refinery_process_model/test_premium_re`

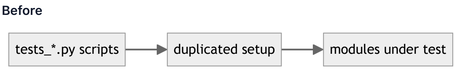
Problem

Verification is spread across many shallow test scripts; the "how to run tests" knowledge lacks **locality**.

Solution

Create a single test runner seam (`pytest`) and express scenarios via fixtures/adapters.

Before



After

If you only do one thing

Do **Candidate A** first: make the main scenario runner work from already-loaded inputs. That will remove a lot of "where are the files?" confusion and will make everything else easier to test and change.